Introduzione ad Haskell

Alessandro Massimo Mermali
alessandroma.mermali@studenti.unicam.it



Cos'è Haskell?

- Linguaggio di programmazione funzionale *puro* basato sul λ-calcolo
- Funzioni di **prima classe** $\rightarrow$ funzioni passate come argomenti, memorizzate, definite anonime
- Funzioni di **ordine superiore** $\rightarrow$ funzioni accettano altre funzioni come argomenti, restituiscono funzioni
- Assenza di **cicli imperativi** (while, for, do-while) $\rightarrow$ uso della ricorsione

LAZINESS E SIDE EFFECTS

- Haskell è pigro (**lazy**) → quando si applica una funzione, l'argomento non viene valutato subito, ma solo se necessario
  La maggior parte dei linguaggi di programmazione sono **zelanti**, valutano gli argomenti prima che la funzione venga invocata
- Assenza di **side effects**
  - Manca il comando di assegnamento
  - Non è possibile modificare la memoria

Tipizzazione

- **Staticamente** tipato → gli errori emergono prima dell'esecuzione
- **Fortemente** tipato → non ci sono conversioni implicite o forzate tra tipi
- Tipi **inferiti** → il compilatore inferisce i tipi automaticamente anche senza dichiararli
  Si può scegliere di scrivere esplicitamente alcuni tipi

GHC

GHC (**Glasgow Haskell Compiler**) è il compilatore di riferimento per Haskell.

Traduce il codice sorgente *.hs* in codice macchina eseguibile.

ghci $\rightarrow$ interprete che fornisce anche un'interfaccia a riga di comando (CLI) dove valutare le espressione Haskell.

Installazione di GHC attraverso GHCup (https://www.haskell.org/ghcup/install/)

Espressioni, valori, tipi e classi

Espressioni e Valori

- Le **espressioni** sono computazioni: `2+5`, `radice 9`
- I **valori** sono i risultati: `7` (da `2+5`), `3` (da `radice 9`)

Tipi e Classi

- I **tipi** sono famiglie di valori (verificabili con `:type`):
  - `Int`: tipo dei numeri interi
  - `Int -> Int`: tipo delle funzioni da intero a intero
- Le **classi** sono famiglie di tipi con caratteristiche comuni:
  - `Int` appartiene a `Num` (numeri), `Ord` (ordinabili), `Show` (stampabili), ecc.

Espressioni e Operatori

Forma	Significato	Esempi
`valore`	numeri, stringhe, ...	`1, "Ciao", True`
`id`	identificatori	`n, somma, print`
`E1 op E2`	operatori	`n - 1`
`id E1 ... En`	applicazione	`sumOp 8`
`(E)`	parentesi	`(n - 1)`

Operatori di uso comune

- **Aritmetici:** `+, -, *, /, ` `` `div` ``, `` `mod` ``
- **Relazioni:** `==, /=, <, <=, >, >=`
- **Connettivi logici:** `&&, ||`
- **Concatenazione (liste/stringhe):** `++`

Applicazione funzionale

Per indicare gli argomenti di una funzione, non si usano le parentesi come in altri linguaggi

```
method(arg1, arg2)
```

ma sono direttamente elencati dopo la funzione stessa.

```
method arg1 arg2
```

L'applicazione funzionale è associativa a sinistra:

- f x y = (f x) y → si applica `f` ad `x`, il risultato viene poi applicato a `y`
- f (x y) → si applica `x` a `y`, poi si applica `f` al risultato

Type checking

Haskell compila solo le espressioni **ben tipate**.
Un'espressione si dice ben tipata quando rispetta una serie di **regole di tipo**. La valutazione di queste regole su un'espressione si dice **type checking**.

Esempi (semplici)

1. `0 :: Int`
   (analogamente per ogni altra costante intera)
2. `"Ciao" :: String`
   (analogamente per ogni altra stringa)
3. **se** `E1 :: Int` **e** `E2 :: Int` **allora** `E1 + E2 :: Int`
   (analogamente per ogni altro operatore su numeri interi)
4. **se** `E1 :: Int` **e** `E2 :: Int` **allora** `E1 <= E2 :: Bool`
   (analogamente per ogni altra relazione applicabile a numeri interi)

Dichiarazioni

Usiamo le dichiarazioni per dare un nome ed un tipo a valori.

```haskell
y :: Int
y = 10
```

Dichiarazione semplice.

```haskell
y :: Int
y = x * x
    where
        x :: Int
        x = 10
```

Dichiarazione locale.

NB: l'identazione è importante
Gli script Haskell sono file `.hs` contenenti una serie di dichiarazioni.

Com'è fatta una funzione?

```
somma10 :: Int -> Int
somma10 x = x + 10
```

- La prima riga associa il tipo `Int → Int` al nome `somma10`
- Il tipo `Int → Int` indica che stiamo definendo una funzione che trasforma valori di tipo `Int` in valori del tipo `Int`.
- La seconda riga definisce il comportamento della funzione quando viene applicata ad un valore `x`.

Funzioni a più argomenti

```
add :: Int -> Int -> Int
add x y = x + y
```

- Le funzioni a più argomenti si scrivono con tante $\rightarrow$ quanto il numero di argomenti.
- Il tipo del risultato è quello scritto dopo l'ultimo $\rightarrow$.
- **Currying:** `Int → Int → Int` = `Int → (Int → Int)`

Regola di tipo

Data una funzione `f` $:: T_1 \to T_2$ e un valore `e` $:: T_1$, allora `f e` $:: T_2$.

Esempi

Dato `addOne :: Int -> Int` e `4 :: Int`. Definire il tipo di `addOne 4`.

GUARDIE

```haskell
pari :: Int -> Bool
pari x | x `mod` 2 == 0 = True
       | otherwise =    False
```

Su Haskell è possibile implementare i controlli `if-else` `if-else` usando le guardie. `otherwise` non è una keyword, ma un alias per `True`.
NB: le guardi controllano *le proprietà* o *le condizioni* dei dati.

Pattern Matching

Il **pattern matching** è un meccanismo che permette di verificare se un valore corrisponde ad una precisa struttura e, in caso positivo, di estrarne i componenti e vincolarli a dei nomi di variabile.

```haskell
raddoppia :: Int -> Int
raddoppia 0 = 0
raddoppia x = 0

testa :: [Int] -> Int
testa (x : _) = x

value :: (Int, String) -> String
value (_, str) -> str
```

TUPLE

Cos'è una tupla?

Una tupla è una sequenza **finita, fissata** e **ordinata** di valori **eterogenei**.

Tipo di una tupla

Il tipo di una tupla con n-componenti è $(T_1, T_2, ..., T_n)$.

```haskell
first :: (Int, Bool) -> Int
first (x, _) = x

second :: (Int, Bool) -> Bool
second (_, y) = y
```

Figure: Applicazione di pattern matching su una tupla

Proiezioni

```haskell
first :: (Int, Bool) -> Int
first (x, _) = x

second :: (Int, Bool) -> Bool
second (_, y) = y
```

In Haskell funzioni come `first` e `second`, che permettono di estrarre elementi da una tupla, si definiscono **proiezioni**.

Le funzioni `fst` e `snd` permettono di estrarre il primo e il secondo elemento da una tupla, a prescindere dal tipo della tupla stessa.

Regole di tipo per le tuple

Formazione

Se $E_1 :: T_1$ e $E_2 :: T_2$, allora $(E_1, E_2) :: (T_1, T_2)$.
Se $E_1 :: T_1, \ldots, E_n :: T_n$, allora $(E_1, \ldots, E_n) :: (T_1, \ldots, T_n)$.

Proiezione

Se $E :: (T_1, T_2)$, allora `fst` $E :: T_1$.
Se $E :: (T_1, T_2)$, allora `snd` $E :: T_2$.

Funzioni a più argomenti con tuple

```haskell
sum1 :: (Int, Int) -> Int
sum (x, y) = x + y

sum2 :: Int -> Int -> Int
sum x y = x + y
```

- `sum1` e `sum2` sono *di fatto* la stessa funzione.
- Il tipo di `sum1` è molto più in linea con ciò che useremo in linguaggio matematico.
- In Haskell è tendenzialmente preferita la prima forma se la funzione accetta una **coppia di numeri interi** e la seconda se può accettare **due numeri interi**.

Tuple particolari

Tuple singole

Non esistono tuple composte da un unico elemento, l'elemento (E) indica semplicemente l'elemento E tra parentesi.
Allo stesso modo, il tipo (T) indica semplicemente il tipo T tra parentesi.

Tuple vuote

Esiste una singola tupla vuota.

- Si scrive `()` ed il suo tipo è `()`.
- Il valore `()` è privo di informazione.
- `()` può avere una sua utilità (similarmente a `void` in Java).

Liste

Cos'è una lista?

Una lista è una sequenza **fissata** e **ordinata** di valori **omogenei**.

Costruttori

- `[]` è una lista vuota
- Se `x` è un elemento di tipo `T` e `xs` è una lista di elementi di tipo `T`, allora `x : xs` è una lista di elementi di tipo `T` che inizia con `x` e finisce con `xs`.

Esempi

Stessa lista scritta in modo diverso:

- $0 : 1 : 2 : []$
- $0 : (1 : (2 : []))$
- $[0, 1, 2]$

Regole di tipo per le liste

Tipo delle liste

$[T]$ è il tipo delle liste con elementi di tipo T.
Dal tipo di una lista **non** si può evincere la sua lunghezza.

Regole di tipo

[] ha tipo $[T]$ per ogni T
se `E` :: T e `F` :: $[T]$, allora `E : F` :: $[T]$

Stringhe

Definizione

Una stringa è una lista di caratteri.

Tipo

`String` e `[Char]` sono lo stesso tipo.

Concatenazione di liste

Operatore

`xs ++ ys` è la lista che contiene tutti gli elementi di `xs` seguiti da tutti gli elementi di `ys`.

Regole di e tipo

Se `E` ::[`T`] e `F` ::[`T`], allora `E ++ F` ::[`T`].
(Notare differenza con regola di tipo per :).

Esercizi sulle tuple

Esercizio 1

Definire una funzione `swap` :: (`Int`, `Int`) $\rightarrow$ (`Int`, `Int`) che, applicata ad una coppia di numeri interi, restituisca la coppia scambiata.

Esercizio 2

Creare una funzione che, dati due punti (`Float`, `Float`), ne calcola la distanza.

Esercizi sulle liste

Pattern Matching sulle liste

- Definire le funzioni lunghezza e somma che calcolano rispettivamente la lunghezza di una lista e la somma di tutti gli interi che vi sono contenuti.

Pattern Matching profondo

- Definire una funzione che, applicata a una lista di coppie di numeri interi, produca la lista dei prodotti degli elementi delle coppie.
- Definire una funzione che, applicata a una lista di numeri interi, produca la lista dei numeri interi pari presenti nella lista.

Esercizi sulle liste

Pattern Matching su più argomenti

- Definire una funzione chiudi che, applicata a due liste `xs` e `ys`, produca la lista degli elementi corrispondenti di `xs` e `ys`.

Pattern Matching sul risultato

- Definire una funzione `apri` che, applicata a una lista di coppie, produca la coppia delle liste degli argomenti.

- Usando l'operatore ++ definire la funzione `inversa` :: `[Int]` → `[Int]` che inverte l'ordine degli elementi di una lista.
- Definire la funzione `ultimo` :: `[Int]` → `Int` per ottenere l'ultimo elemento di una lista (se presente).

Sequenza di fibonacci

- Si scriva una funzione `fibonacci i` che calcoli l'i-esimo elemento della sequenza di fibonacci.

SEQUENZA DI FIBONACCI - SOLUZIONE

- Si scriva una funzione `fibonacci i` che calcoli l'i-esimo elemento della sequenza di fibonacci.

```haskell
fibonacci :: Int -> Integer
fibonacci 0 = 0
fibonacci 1 = 1
fibonacci x = fibonacci (x - 1) + fibonacci (x - 2)

fibonacci :: Int -> Integer
fibonacci x = aux 0 1 k
    where
        aux m _ 0 = m
        aux m n k = aux n (m + n) (k - 1)
```

Teorema

Sia A la matrice $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, allora $A^k = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$ per ogni $k > 0$.

Dimostrazione per induzione

Se $k = 1$ abbiamo $A^1 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$

Se $k > 1$ abbiamo

$$
\begin{aligned}
A^k &= A \times A^{k-1} && \text{definizione di } A^k \\
&= A \times \begin{pmatrix} F_k & F_{k-1} \\ F_{k-1} & F_{k-2} \end{pmatrix} && \text{ipotesi induttiva} \\
&= \begin{pmatrix} F_k + F_{k-1} & F_{k-1} + F_{k-2} \\ F_k & F_{k-1} \end{pmatrix} && \text{prodotto matriciale} \\
&= \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix} && \text{sequenza di Fibonacci}
\end{aligned}
$$

Osservazione

Se riusciamo a trovare un modo efficiente per calcolare A^k, allora c'è un modo efficiente per calcolare il numero di fibonacci con indice k.

Soluzione

$$A^k = \begin{cases} I & \text{se } k = 0 \\ B \times B & \text{se } k > 0 \text{ è pari, dove } B = A^{k/2} \\ A \times A^{k-1} & \text{se } k \text{ è dispari} \end{cases}$$